DIGITAL NURTURE 5.0

Angular (v20+) Hands-On Lab Report

Incremental Project: Student Course Portal

Hands-On:	Hands-On 8 [Advanced-Professional]
Topic:	HTTP Client, JSON Server CRUD, RxJS Operators & Interceptors
Developer:	Cathy George
OS Version:	Windows 11
Date:	July 22, 2026
Track:	Java FSE Angular Track

1. Objective

The primary objective of this exercise is to integrate robust HTTP client-server communication using Angular's HttpClient. By launching a local JSON Server instance backend that stores course records in db.json, the application performs full CRUD database syncs dynamically. Additionally, we integrate advanced RxJS pipelined operators including map, tap, switchMap, retry, and catchError to manipulate data flows and recover from connection faults. Finally, we implement functional HttpInterceptors to manage application-wide loading indicator bars and global API error intercepting coupled with user notification toast popups.

2. Angular Concepts Covered

• **HttpClient:** Injectable Angular service enabling backend REST API operations over XMLHttpRequests.

• **JSON Server backend:** Mock database utility providing instant, local REST endpoints dynamically sync'd from db.json.

• **RxJS Operators:** Utilizing map to scrub string inputs, tap for side effect caching, switchMap to chain operations, retry for network fault resilience, and catchError for exception intercepts.

• **HttpInterceptorFn:** Modern functional interceptor configuration capturing outbound requests and inbound responses to inject global behavior.

• **Loading Spinner Service:** Tracking active HTTP request counts and overlaying full-page blur blocks when pending operations are running.

• **Global Error Handling:** Intercepting status code failures (404, 500, etc.) in a unified interceptor pipeline and surfacing detailed diagnostic alerts via toast banners.

3. Course Service API Operations (src/app/services/course.service.ts)

```
// Fetch and Refresh courses list from JSON Server
refreshCourses(): Observable<Course[]> {
return this.http.get<Course[]>(this.apiUrl).pipe(
retry(2),
map(courses => courses.map(c => ({ ...c, title: c.title.trim() }))),
tap(courses => this.coursesSubject.next(courses)),
catchError(this.handleError)
);
}

// Add new Course record to JSON Server
createCourse(course: Omit<Course, 'id' | 'enrolled'>): Observable<Course[]> {
return this.http.post<Course>(this.apiUrl, { ...course, enrolled: false }).pipe(
tap(created => console.log('Created course:', created.code)),
switchMap(() => this.refreshCourses()),
catchError(this.handleError)
);
}
```

4. Http Interceptors

Functional Loading Interceptor (src/app/interceptors/loading.interceptor.ts)

```
export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
const loadingService = inject(LoadingService);
loadingService.show();
return next(req).pipe(
delay(1000), // artificial delay for visual demo
finalize(() => loadingService.hide())
);
};
```

Functional Error Interceptor (src/app/interceptors/error.interceptor.ts)

```
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
const notificationService = inject(NotificationService);
return next(req).pipe(
catchError((error: HttpErrorResponse) => {
let errorMessage = 'An unknown error occurred!';
if (error.error instanceof ErrorEvent) {
errorMessage = `Client-side error: ${error.error.message}`;
} else {
errorMessage = `Server-side error (${error.status}): ${error.statusText || 'Unable to connect'}`;
}
notificationService.show(errorMessage, 'warning');
return throwError(() => new Error(errorMessage));
})
);
};
```

5. Verification and Screen Outputs

The HttpClient operations and Interceptor loading/error transitions were successfully verified:

[Screenshot Placeholder]: Loading Spinner Overlay - A visual translucent screen overlay with a spinning wheel and 'Loading Academic Data...' text halts interactions during active HTTP GET operations.

[Screenshot Placeholder]: Database Synchronized List - Catalog loads from JSON Server REST API (port 3000) and displays course cards for computer science, IT, and design tracks.

[Screenshot Placeholder]: Add/Delete Course Operations - Submitting the add course form performs a POST request to JSON Server. Clicking delete makes a DELETE call, followed by an automatic reactive list refetch.

[Screenshot Placeholder]: Error Toast Notification - Shutting down the JSON server backend triggers the HTTP catchError pipeline, intercepting the failed connection and broadcasting an warning toast banner: 'Server-side error (0): Unable to connect to server'.

6. Learning Outcomes & Conclusion

✓ Configured provideHttpClient with functional interceptors in a standalone boot configuration.

✓ Created an active-request loading counter service to block UI input during network roundtrips.

✓ Employed retry(2) and catchError to inject fault tolerance in API handlers.

✓ Integrated client CRUD methods mapping directly to REST paths on JSON Server.

✓ Decoupled notification broadcasts using a custom global Toast state emitter.

Conclusion

Hands-On 8 establishes robust asynchronous backend service communication for the Student Course Portal. Using standard RxJS patterns, functional interceptors, and a local mock REST API server, data operations now reflect database-persisted states while rendering loading animations and error messages.